Failure Handling - Cheat Sheet

In distributed systems, failure is the default. Networks drop packets. Services restart. Disks fill up. Dependencies time out. Designing for failure is about absorbing failures without taking the rest of the system down with you. Prevention only goes so far once you have more than one process talking over a network.

Common failure modes

Real failures fall into a small set of shapes.

Mode	What it looks like
Timeout	Request takes too long, caller gives up
Connection refused	Service isn't accepting connections
5xx errors	Service is up but returning errors
429 / rate limit	Caller exceeded their quota
Slow response	No error, but latency climbs
Partial success	Some operations succeed, others fail
Cascading failure	One service's failure overloads its callers
Split brain	Network partition makes the system disagree with itself
Poison message	A bad input keeps crashing consumers

The patterns below are tools for handling these.

Timeouts and deadlines

Every external call needs a deadline. Without one, a slow dependency can hold your threads forever and eventually exhaust your pool, taking the service down with it.

```java
// HTTP timeouts in Java
HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(2))
    .build();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.example.com/users/42"))
    .timeout(Duration.ofSeconds(5))
    .build();
```

Two distinct timeouts:

- Connect timeout: how long to wait for the TCP handshake.
- Request timeout: total time the request can take.

Deadline propagation

In a service chain (A -> B -> C), the deadline should flow with the request. If A has 5 seconds total and B's call took 1s, B should allow C only 4 seconds.

gRPC propagates deadlines automatically through `Context`. HTTP usually needs a custom header (`X-Deadline-Ms` or similar).

```java
// gRPC propagates deadlines via stub
stub.withDeadlineAfter(2, TimeUnit.SECONDS).getUser(request);
```

Rule of thumb: every external call has a timeout. Every timeout is shorter than the caller's timeout. Otherwise retries pile up and threads run out.

Retries

Networks blip. Some failures are transient. Retrying is the simplest fix, with sharp edges.

When to retry

- Only on idempotent operations, or with an idempotency key.
- Only on retryable errors: network failures, 503, 429, gRPC `UNAVAILABLE` or `DEADLINE_EXCEEDED`.
- Never on 400, 401, 403, 404. Retrying won't fix them.

Exponential backoff with jitter

```
long delay = 200;
for (int attempt = 0; attempt < 5; attempt++) {
    try {
        return callService();
    } catch (RetryableException e) {
        if (attempt == 4) {
            throw e;
        }
        long jitter = ThreadLocalRandom.current().nextLong(delay);
        Thread.sleep(delay / 2 + jitter); // half base + jitter
        delay = Math.min(delay * 2, 10_000); // cap at 10s
    }
}
```

Why each piece matters:

- Exponential: quick at first, then slows down so failed services get breathing room.
- Cap: stops the delay growing forever.
- Jitter: random offset. Without it, all clients retry at the same instant and bury the recovering service. Always add jitter.

Production code should use a library: Resilience4j, Spring Retry, or Failsafe in Java. Roll your own only if you have to.

Retry budget

A retry is a free amplifier: 5 retries means 5x the load. When the downstream service is sick, retries pile on and make recovery harder. Defenses:

- Cap retries per call (3 is usually enough).
- Cap total retries per second across the service (a “retry budget”).
- Skip retries when the circuit is open.

Circuit breakers

Wrap a flaky dependency. After enough failures, the breaker opens and short-circuits future calls. New calls fail immediately instead of waiting on the timeout.

States:

State	Behavior
Closed	Calls go through. Failures are counted.
Open	Calls fail immediately without hitting the dependency.
Half-open	A few test calls go through. Success closes the breaker. Failure reopens it.

```
// Resilience4j example
CircuitBreaker breaker = CircuitBreaker.ofDefaults("user-service");

Supplier<User> decorated = CircuitBreaker
    .decorateSupplier(breaker, () -> userClient.find(id));

try {
    User user = decorated.get();
} catch (CallNotPermittedException e) {
    // circuit is open, fall back
    return cachedUser(id);
}
```

Key tuning knobs:

- Failure rate threshold: percentage of failures that opens the breaker (often 50%).
- Sliding window: how many recent calls count toward the rate.
- Wait duration: how long to stay open before trying half-open.
- Slow call duration: treat slow calls as failures so the breaker reacts to brownouts, not just outages.

Without circuit breakers, slow dependencies eat your threads. With them, you fail fast and shed load before things spiral.

Bulkheads

Isolate failures by partitioning resources. Like watertight compartments on a ship, a failure in one section doesn't sink the whole vessel.

In practice:

- Separate thread pools per dependency. A flooded user-service pool doesn't starve calls to order-service.
- Separate connection pools per upstream.
- Separate Kubernetes deployments per critical workload, so resource exhaustion in one doesn't take out another.

```
// Resilience4j bulkhead
ThreadPoolBulkhead bulkhead = ThreadPoolBulkhead.ofDefaults("user-service");

CompletionStage<User> future = bulkhead.executeSupplier(() -> userClient.find(id));
```

Configure max concurrent calls and a queue size. Calls beyond the queue fail fast.

Fallbacks and graceful degradation

When a dependency fails, return something useful in place of an error.

```
public User getUser(long id) {
    try {
        return userClient.find(id);
    } catch (Exception e) {
        log.warn("user-service down, returning cached", e);
        return cache.get(id, this::placeholderUser);
    }
}
```

Common fallback strategies:

- Cached value: return the last known good response.
- Default value: return a generic empty object.
- Reduced functionality: drop personalization, show the static page.
- Asynchronous: queue the work for later instead of failing now.

Fallback choices belong to product as much as engineering. Get them aligned before the outage forces the decision.

Idempotency

A retry can't be safe unless the operation is idempotent: running it twice has the same effect as running it once. GET, PUT, DELETE are idempotent by HTTP spec. POST is not.

For non-idempotent operations (creating an order, charging a card), use an idempotency key:

```
POST /orders
Idempotency-Key: 8f3a-2bc4-...

{ "items": [...] }
```

Server-side flow:

1. Receive the request.
2. Check the idempotency key in a store (Redis, DB).

3. If a record exists, return the stored response.
4. Otherwise, do the work and store the response under that key.

The key needs to live long enough to cover all possible retries (usually 24 hours). Stripe's API is the canonical reference.

Dead letter queues

For async work (Kafka, SQS, RabbitMQ), some messages can't be processed. They might be malformed, reference deleted resources, or trigger bugs in the consumer.

Without handling, a poison message blocks the queue. With a DLQ, the broker sends repeated failures to a separate queue for human investigation.

Typical setup:

- Retry the message a few times with backoff.
- After max retries, route to the DLQ.
- Alert on DLQ depth.
- Build tooling to inspect, fix, and replay messages.

A DLQ that nobody watches is a slow leak. Set up a dashboard.

Outbox pattern

A common problem: you need to update the database AND publish an event. Doing them as separate operations means one can fail without the other, leaving the system inconsistent.

The outbox pattern:

1. In one DB transaction, write the business data AND insert a row into an outbox table.
2. A separate process (or CDC tool like Debezium) reads new outbox rows and publishes them to the message broker.
3. After successful publish, mark the row sent or delete it.

```
BEGIN;
INSERT INTO orders (...) VALUES (...);
INSERT INTO outbox (aggregate, event_type, payload) VALUES (...);
COMMIT;
```

Both succeed or both fail. Atomicity guarantees consistency without distributed transactions.

Saga pattern

Distributed transactions are hard. Two-phase commit doesn't scale. The Saga pattern replaces them with a sequence of local transactions, each with a compensating action that runs if a later step fails.

Example: an order saga.

1. Reserve inventory -> compensation: release inventory
2. Charge payment -> compensation: refund payment
3. Schedule shipment -> compensation: cancel shipment

If step 3 fails, the saga runs compensations in reverse: cancel any partial shipment work, refund the payment, release the inventory.

Two flavors:

- Choreography: each service reacts to events. No central coordinator. Simple at first. Hard to debug as the saga grows.
- Orchestration: a central service drives the saga. Easier to trace and modify. Adds a coordinator dependency.

For complex sagas, use a workflow engine: Temporal, Cadence, AWS Step Functions, Camunda.

Health checks

Probes that tell the platform whether your service is okay.

Three flavors map to Kubernetes probes:

- Liveness: is the process alive? Used to decide when to restart.
- Readiness: is the process ready to take traffic? Used to gate load balancer routing.
- Startup: has the process finished starting? Used to give slow boots a longer grace period.

Common mistakes:

- Liveness probes that hit the DB. A slow DB triggers a restart loop, making it worse.
- Readiness probes that return 200 before the cache warms up. New pods serve cold-cache traffic and look unhealthy.
- Health endpoints that aggregate every dependency. One slow dependency makes the whole service look down.

Keep liveness cheap. Make readiness honest. Separate the two endpoints.

Observability for failures

You can’t fix what you can’t see.

- Metrics: error rate, latency histograms (p50, p95, p99), retry counts, circuit breaker state changes, DLQ depth, saturation per pool.
- Logs: structured, with correlation IDs that propagate across services. Log enough context to reproduce the failure.
- Traces: distributed traces showing the path of a request through services. OpenTelemetry is the standard.

- Alerts: alert on symptoms users feel: error rate, latency. Internal metrics belong on dashboards, not pagers.

The “Four Golden Signals” from Google’s SRE book: latency, traffic, errors, saturation. Track these for every service.

Best practices

- Every external call has a timeout. No exceptions.
- Retries are amplifiers. Cap them per call and per service.
- Always add jitter to backoff. Synchronized retries kill recovering services.
- Treat slow calls as failures. A service that hangs is just as bad as one that errors.
- Circuit breakers protect both sides. They shed load from a failing dependency and free up resources in the caller.
- Design for idempotency. Either make the operation safe to retry, or use idempotency keys.
- Compose patterns: timeout + retry + circuit breaker + bulkhead is a typical layered defense.
- Test failure paths. Chaos engineering, kill-switch tests, dependency outages in staging. The path you never exercise is the one that breaks.
- Don’t aggregate health checks. Separate liveness, readiness, and per-dependency health.
- Plan the degraded experience. Decide with product what to show when a dependency is down.

Common gotchas

- Retries on top of retries. Service A retries 3x. Service B (called by A) also retries 3x. The downstream sees 9 attempts per logical request. Coordinate retry layers.
- No timeout means infinite wait. Default HTTP clients in some languages have no request timeout. Always set one.
- Open circuit + bad fallback = silent failure. If your fallback returns stale or wrong data, users see broken behavior with no error. Log and meter every fallback.
- DLQ as a graveyard. If nobody monitors the DLQ, messages pile up forever and the data is lost in practice. Alert on depth.
- Idempotency key reuse. If clients reuse a key with different parameters, you’ll return stale results for a new request. Hash the request body and reject mismatches.
- Cascading retries. A retry storm from one service can take down the entire chain. Use jitter, budgets, and circuit breakers together.
- Saga rollback isn’t transactional. Compensations can themselves fail. Design them to be idempotent and retriable.
- Liveness probes too aggressive. Tight timeouts and low failure thresholds restart healthy pods during a normal GC pause.
- Hot loops on transient errors. Retrying with no delay (or very short delay) burns CPU and pounds the dependency. Always backoff.

- “Exactly once” is a myth. True exactly-once delivery doesn’t exist in distributed systems. Design for at-least-once delivery with idempotent consumers.

Quick reference

Pattern	Use it for
Timeouts	Every external call. Bound the wait.
Deadline propagation	Pass the remaining time budget down the call chain.
Retries	Transient failures on idempotent calls.
Exponential backoff + jitter	Avoid synchronized retry storms.
Circuit breaker	Fail fast when a dependency is broken.
Bulkhead	Isolate resources per dependency.
Fallback	Return useful degraded data in place of an error.
Idempotency key	Make non-idempotent operations safe to retry.
Dead letter queue	Quarantine poison messages from async pipelines.
Outbox pattern	DB update + event publish atomically.
Saga	Distributed transactions with per-step compensation.
Health checks	Tell the platform when to restart or route traffic.
Four Golden Signals	Latency, traffic, errors, saturation.